
Setting Up a New Instance

Deploying Flipdeck for another company

Who this is for	Someone deploying a separate, independent copy of Flipdeck
Time needed	Roughly two hours, most of it waiting on account signups
Technical level	You need to run a few commands. No coding required.
Result	A completely separate app, database and files. Your instance is untouched.

This guide sets up a NEW instance for a different company. It does not transfer or affect the existing Foundational Real Estate system, which keeps running exactly as it is.

What you are about to do

Flipdeck is one codebase plus four external services. A new instance means the same code pointed at that company's own services, with their own database. Nothing is shared between instances.

Step	What happens	Roughly
1. Copy the code	Put the code in the new company's GitHub account	10 min
2. Create four accounts	Database, file storage, email, hosting	45 min
3. Configure settings	Paste 14 values into the hosting dashboard	20 min
4. Deploy and migrate	Build the site, create the database tables	15 min
5. Create the first Owner	One command. Without this nobody can sign in.	5 min
6. Brand and invite	Company name, logo, add the team	15 min

The single most important step is number 5.

Sign-in is invite-only: the app only emails a sign-in link to an address that already has an account, and creating accounts requires being signed in. On a brand-new database nobody exists, so nobody can ever get in. The bootstrap command breaks that deadlock. Skip it and the new instance is unusable.

What stays separate

Each instance is fully isolated. There is no shared tenancy, no shared database, and no way for one company to see another's data.

Thing	Shared between instances?
Source code	Yes, the same code. Improvements can be pulled into either.
Database	No. Separate database, separate everything in it.
Uploaded files	No. Separate storage bucket.
User accounts and logins	No.
Costs and billing	No. Each company pays for its own services.

What it costs to run

All four services have free tiers, and a single-company property portfolio is small data. Expect the free tiers to cover it initially. Check current pricing when you sign up, since these change.

Service	What it does	Typical starting tier
Neon	PostgreSQL database	Free tier

Service	What it does	Typical starting tier
Vercel	Runs and hosts the app	Free for personal, paid for commercial use
Cloudflare R2	Stores photos and receipts	Free tier, generous
Resend	Sends sign-in emails	Free tier covers low volume
Domain name	Optional. A vercel.app address works fine.	Registrar's annual fee

Vercel's free tier is licensed for non-commercial use. A company running this as a business tool should be on a paid plan. Confirm the current terms.

Step 1

Copy the code

The new company needs their own copy of the repository, so they control it and can receive updates without touching yours.

Easiest route: on GitHub, open the existing repository and use **Fork** into the new company's account. If they would rather have a clean history, or the original should stay private, clone and push instead:

```
git clone https://github.com/arահfranco/flipdeckapp.git newcompany-flipdeck
cd newcompany-flipdeck
git remote set-url origin https://github.com/THEIR-ACCOUNT/THEIR-REPO.git
git push -u origin main
```

Nothing secret lives in the repository.

Passwords, API keys and database addresses are all kept in settings, never in the code. The .env file that holds them locally is deliberately excluded from Git. Copying the code shares no credentials.

Step 2

Create the four accounts

Each is signed up for in the new company's name. Collect the values listed in the right-hand column as you go, you will paste them all in at Step 3.

Neon, the database

- Sign up at neon.tech and create a project.
- From the connection details, copy the **pooled** connection string. This is DATABASE_URL.
- Copy the **direct** (unpooled) connection string too. This is DIRECT_URL, used for database migrations.

Cloudflare R2, the file storage

- In the Cloudflare dashboard, open R2 and create a bucket. The name is yours to choose; note it as R2_BUCKET_NAME.
- In the bucket's Settings, enable the **Public Development URL** and copy it. This is R2_PUBLIC_URL, and it is how uploaded photos are displayed.
- Create an R2 API token with **Object Read and Write** permission. It shows the access key and secret **once**. Copy both immediately.
- Your Cloudflare account ID is R2_ACCOUNT_ID, a 32-character hexadecimal string.

No CORS configuration is needed.

Uploads are sent to the app's own server, which passes them to storage. Older setups had the browser talk to Cloudflare directly, which required a CORS rule and failed in ways that were extremely hard to diagnose. That is no longer the case. If a guide tells you to add a CORS policy, it is out of date.

Resend, the sign-in emails

- Sign up at resend.com and create an API key. It begins with `re_`. This is `EMAIL_SERVER_PASSWORD`.
- Verify the company's sending domain, then use an address at that domain as `EMAIL_FROM`.
- Until a domain is verified, Resend only delivers to the account holder's own address. Sign-in links to anyone else will silently not arrive.

Vercel, the hosting

- Sign up at vercel.com and import the GitHub repository from Step 1.
- Do not deploy yet. Add the settings in Step 3 first, otherwise the first build produces a broken site.

Step 3

Configure the settings

In Vercel, go to Settings, then Environment Variables, and add each of these. Tick Production, Preview and Development for every one.

Setting	What to put in it
<code>DATABASE_URL</code>	Neon pooled connection string
<code>DIRECT_URL</code>	Neon direct connection string
<code>AUTH_SECRET</code>	A long random string. Generate one, see below.
<code>NEXTAUTH_SECRET</code>	The same value again, under the older name.
<code>EMAIL_SERVER_HOST</code>	<code>smtp.resend.com</code>
<code>EMAIL_SERVER_PORT</code>	<code>587</code>
<code>EMAIL_SERVER_USER</code>	<code>resend</code>
<code>EMAIL_SERVER_PASSWORD</code>	The Resend API key, beginning <code>re_</code>
<code>EMAIL_FROM</code>	Company Name < <code>noreply@theirdomain.com</code> >
<code>R2_ACCOUNT_ID</code>	Cloudflare account id, 32 hex characters
<code>R2_ACCESS_KEY_ID</code>	R2 access key, 32 characters
<code>R2_SECRET_ACCESS_KEY</code>	R2 secret, 64 characters
<code>R2_BUCKET_NAME</code>	The bucket name chosen in Step 2

Setting	What to put in it
R2_PUBLIC_URL	The bucket's public development URL

To generate the secret, run this and paste the output into both AUTH_SECRET and NEXTAUTH_SECRET:

```
openssl rand -base64 32
```

Never paste quote marks.

In a .env file values are written inside quotes, and the file format strips them. Vercel does not. It stores exactly what you paste, quotes included. Paste **flipdeck**, never **"flipdeck"**. A quoted value is not empty, so it passes every check for a missing setting and then fails much later as something apparently unrelated. This exact mistake cost three separate debugging sessions on the original build.

NEXTAUTH_URL is deliberately absent from that list. Leaving it unset lets the app build sign-in links from whichever address the user actually visited, which is what you want when a custom domain is added later.

Step 4

Deploy and create the database tables

Deploy from the Vercel dashboard. The build should succeed, but the site will not work yet: the database is empty and has no tables.

On your own machine, in the project folder, create a file named **.env** containing the two database lines, then create the tables:

```
DATABASE_URL="the pooled connection string"
DIRECT_URL="the direct connection string"
```

```
npm install
npx prisma migrate deploy
```

That creates every table in the correct order. Quote marks are fine here, in the .env file they are stripped. It is only the Vercel dashboard where they cause harm.

Do not run npm run db:seed.

The seed file fills the database with demonstration data: four sample properties in Ohio, fictional staff, and the original company's name and email addresses. It exists for local development. Running it against a real new instance gives the client someone else's data to delete by hand.

Step 5

Create the first Owner

This is the step that makes the instance usable. Run it once, with the same .env file in place:

```
npm run bootstrap "Acme Property Group" owner@acme.com "Jane Doe"
```

Three values, in order: the company name, the first Owner's email address, and their full name. It creates the company record and that one account, then stops.

The command refuses to run if the database already has any users, so it cannot be used to give someone an Owner account on a system that is already live.

Why this is necessary

The app only emails a sign-in link to an address that already has an account, so an unknown address gets nothing. Creating accounts is behind the login. On an empty database those two rules deadlock, and this command is the only way through.

Step 6

Brand it and invite the team

- Go to the site and sign in with the email address from Step 5.
- Open **Settings** and set the company name, tagline, contact details and logo. These appear on the sign-in screen and throughout the app. Nothing is hardcoded, so there is no code to edit.
- Open **Settings, Users** and add the rest of the team with the appropriate roles.
- Add their first property and confirm the budget checklist appears.

Choosing roles

Role	Suited to
Owner	Full access, including user management and company settings.
Partner	Investors. Sees properties, expenses and capital. No payroll.
Partner (Lender)	Lenders. Sees properties and capital only. No expense detail.
Bookkeeper	Sees expenses, bank and payroll. No partner capital.

Bookkeepers see payroll but not capital; Partners see capital but not payroll. The asymmetry is intentional.

Optional

Add a custom domain

A vercel.app address works permanently and needs nothing. To use the company's own domain, a subdomain is usually best, as it leaves any existing website alone.

- In Vercel, Settings, Domains, add for example app.theircompany.com

- At their DNS provider add a CNAME record: name **app**, value **cname.vercel-dns.com**
- Vercel issues the HTTPS certificate automatically, usually within minutes.

No other setting needs changing. Sign-in links follow whichever address was used, and uploads are unaffected because they go through the app's own server.

Before you hand it over

Work through this list on the live site. Each item exercises a different service, so between them they prove the whole system is wired up.

Check	Proves
Sign in with a link sent to your email	Database, email delivery and login all work
Create a property	The database accepts writes
Confirm the budget checklist appears on it	Budget lines are created automatically
Upload a photo to that property	File storage credentials are correct
Log an expense against a rehab subcategory	Expense logging works
Check the budget tab shows that amount as Actual	The rollup calculation is running
Add a bank account and one money-in transaction	Cash tracking works
Open Company Value	The balance sheet calculates
Invite a second user and have them sign in	Invitations work for other people, which is where an unverified email domain shows up
Delete the test property	Cascading delete works, and clears your test data

If something fails

Symptom	Almost always
File storage is not configured, missing R2_...	Those settings are absent from the deployment, or it was not rebuilt afterwards.
Please use AWS4-HMAC-SHA256	The R2 access key or secret is wrong. They should be 32 and 64 characters.
No sign-in email arrives	Either the address has no account, or the Resend sending domain is not verified. An unverified domain delivers only to the Resend account holder.
Server error after clicking a sign-in link	AUTH_SECRET is missing.
Build fails: URL must start with postgresql://	DATABASE_URL has quote marks around it in Vercel.
A setting was changed but nothing happened	Settings are read when the site is built. Redeploy.

Settings apply at build time.

A running deployment can only see settings that existed when it was built. After changing any value in Vercel you must redeploy: Deployments, newest, Redeploy. This catches almost everyone at least once.

Keeping the instance up to date

Because the new instance is a copy of the same codebase, improvements made to one can be pulled into the other. In the new company's repository, add the original as a second remote and merge from it when you want the changes:

```
git remote add upstream https://github.com/arահfranco/flipdeckapp.git
git fetch upstream
git merge upstream/main
git push
```

Pushing deploys automatically. If an update includes a database change, run **`npx prisma migrate deploy`** against their database **before** the new code goes live. Schema changes are written additively, so the old code keeps working against the new database; the reverse is not true, and new code reading a column that does not exist fails immediately.

What to hand over

- The site address, and confirmation the first Owner can sign in.
- Logins for the four service accounts, ideally created in the company's own name from the start so nothing needs transferring later.
- The main documentation PDF, whose user guide section covers day-to-day use.
- Who to contact when something breaks, and on what terms.

Flipdeck new instance guide. Reflects the codebase as of 22 July 2026. Setting up a new instance does not affect any existing instance.